BioSpecInfo

Interface Reference — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v240** · commit `6794af6`

Generato il 2026-11-05

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Interface Reference — BioSpecInfo

Field	Value
Software	BioSpecInfo
Version described	`bsi-v240`
Purpose	Document the project's HTTP interfaces and the external APIs the application consumes.

0. A necessary premise

BioSpecInfo exposes no API. It is an entirely client-side Progressive Web App, served as static files: there is no backend, no application server, no REST endpoints belonging to the product.

This document would therefore be empty, were it not for two interfaces that do exist and that an assessor is entitled to see documented:

	What it is	Documented in
Spectra proxy	An **optional** Cloudflare Worker, owned by the user, that forwards calls to the AI providers. It is the only HTTP interface written in this project.	§1, §2
External APIs consumed	Public third-party services the application queries, always optionally.	§3
Internal JavaScript APIs	The extension points of the code, for whoever has to work on it.	§4

> An OpenAPI specification of non-existent endpoints would be plausible and false documentation. Only what exists is described here.

1. Spectra proxy — overview

The proxy solves two problems that have no other solution from the browser:

1. **CORS.** Several AI providers do not send the headers that allow a web page

to call them directly. From the Worker the call originates from a server and the constraint does not apply.

1. **Custody of the keys.** With the proxy the API key lives among the Worker's

secrets and not on the user's device.

It is **optional**: without a proxy the application works using the keys the user enters locally.

Deploy

```
cd proxy
npx wrangler login
npx wrangler deploy
npx wrangler secret put GROQ_KEYS     # one or more keys, comma-separated
```

Configuration

Variable	Type	Default	Description
`ORIGINI`	secret/variable	*(empty = all)*	List of permitted origins, comma-separated. **Leaving it empty makes the proxy open to anyone:** not advisable.
`LIMITE_IP`	variable	`20`	Requests per single IP within the time window
`TETTO_GIORNO`	variable	`2000`	Total daily requests
`GROQ_KEYS`, `GEMINI_KEYS`, `ANTHROPIC_KEYS`, `OPENAI_KEYS`, `XAI_KEYS`, `NVIDIA_KEYS`, `ZAI_KEYS`, `DEEPSEEK_KEYS`	secrets	—	API keys. Several keys comma-separated: when the first one's quota is exhausted (429), the proxy moves to the next **within the same request**.

> The variable names are kept in Italian because that is what they are called in the Worker: translating them here would document a configuration that does not exist.

Endpoints

Method	Path	Description
`OPTIONS`	`/*`	CORS preflight
`GET`	`/stato`	Which providers have a key configured
`GET` `POST`	`/{provider}/{path...}`	Forwarding to the provider

Recognised providers: `anthropic` · `gemini` · `groq` · `xai` · `nvidia` · `zai` · `openai` · `deepseek`

Status codes

Code	Meaning
`200`	The provider's answer, forwarded as it is
`204`	CORS preflight accepted
`400`	Request body unreadable
`403`	Origin not permitted by the `ORIGINI` list
`404`	Unknown provider
`405`	Method other than GET, POST or OPTIONS
`429`	Per-IP limit or daily ceiling exceeded

Code	Meaning
`503`	No key configured for that provider

Security choices

- **No header is forwarded blindly.** Only `anthropic-version` and

`anthropic-beta` pass through; any credential sent by the client stays out. A proxy that forwards everything is an open relay.

- **The `key` parameter is not propagated.** If a client puts it in the query

string, the proxy discards it: the key is the server's.

- **The body is read once and kept**, because retrying with a different key

requires being able to send it again — and a stream is consumed.

---

2. OpenAPI 3.0 specification

> The `description` fields are translated here; the **error messages in the examples are left in Italian**, because that is what the proxy actually returns to a user of the Italian application. Translating them would document responses that do not exist.

```yaml
openapi: 3.0.3
info:
  title: Spectra Proxy
  version: "1.0.0"
  description: |
    Optional proxy (Cloudflare Worker) for calls to language-model providers.
    It removes CORS blocks and keeps the API keys on the server rather than on
    the user's device.

    It is not part of the BioSpecInfo application: it is published by the user
    on their own Cloudflare account, and the application uses it only if it
    knows its address.
  license:
    name: Proprietary — All rights reserved (Samuele Pio Provenzano)
servers:
  - url: https://{workername}.{account}.workers.dev
    description: The user's instance
    variables:
      workername: { default: spectra-proxy }
      account:    { default: example }

tags:
  - name: Status
    description: Querying the configuration without revealing secrets
  - name: Forwarding
    description: Forwarding requests to the model providers

paths:
  /stato:
    get:
      tags: [Status]
      summary: Configured providers
      description: |
        Lists the providers for which the proxy holds at least one key.
        It reveals no key. The application uses it to show only the models that
        actually work, instead of letting them fail one by one.
      responses:
        "200":
          description: List of active providers
          content:
            application/json:
              schema:
                type: object
                required: [fornitori]
                properties:
                  fornitori:
                    type: array
                    items:
                      type: string
                      enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
              examples:
                twoKeys:
                  summary: Two providers configured
                  value: { fornitori: [groq, gemini] }
                none:
                  summary: Worker published but with no secrets
                  value: { fornitori: [] }
        "403":
          $ref: "#/components/responses/OrigineNonAmmessa"
```

```
/{fornitore}/{percorso}:
  parameters:
    - name: fornitore
      in: path
      required: true
      description: Destination provider
      schema:
        type: string
        enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
    - name: percorso
      in: path
      required: true
      description: |
        Path of the endpoint at the provider, reassembled as it is.
        Examples: `openai/v1/chat/completions` for Groq,
        `v1beta/models/gemini-3.6-flash:streamGenerateContent` for Gemini.
      schema: { type: string }
      example: openai/v1/chat/completions
  post:
    tags: [Forwarding]
    summary: Forward a request to the provider
    description: |
      The body is transmitted unchanged. The proxy adds the key in the way
      that provider requires (an `Authorization: Bearer` header, `x-api-key`,
      or a query parameter) and, if the call fails for exhausted quota,
      retries with the next key within the same request.
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            description: Body expected by the provider's API, transmitted unchanged
          examples:
            groq:
              summary: Streaming completion on Groq
              value:
                model: llama-3.3-70b-versatile
                stream: true
                messages:
                  - role: user
                    content: Which IR bands identify a carboxylic acid?
    responses:
      "200":
        description: |
          The provider's answer, forwarded without modification (typically
          `text/event-stream` when `stream: true`).
        content:
          application/json:
            schema: { type: object }
          text/event-stream:
            schema: { type: string }
      "400": { $ref: "#/components/responses/CorpoIlleggibile" }
      "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
      "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
      "429": { $ref: "#/components/responses/TroppeRichieste" }
      "503": { $ref: "#/components/responses/ChiaveAssente" }
  get:
    tags: [Forwarding]
```

```
      summary: Forward a read request to the provider
      description: |
        It serves, among other things, the resolution of the Gemini model, which
        queries `ListModels` with GET and could not pass without a key in the
        browser.
      responses:
        "200":
          description: The provider's answer
          content:
            application/json:
              schema: { type: object }
        "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
        "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
        "429": { $ref: "#/components/responses/TroppeRichieste" }
        "503": { $ref: "#/components/responses/ChiaveAssente" }

components:
  schemas:
    Errore:
      type: object
      required: [errore]
      properties:
        errore:
          type: string
          description: Message in Italian, meant to be shown to the user
  responses:
    OrigineNonAmmessa:
      description: The request's origin is not in the `ORIGINI` list
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Origine non ammessa da questo proxy." }
    FornitoreSconosciuto:
      description: The first path segment does not match a known provider
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Fornitore sconosciuto: pippo" }
    CorpoIlleggibile:
      description: The request body could not be read
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Corpo della richiesta illeggibile." }
    TroppeRichieste:
      description: The per-IP limit or the daily ceiling has been exceeded
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Troppe richieste da questo indirizzo: riprova fra un minuto." }
    ChiaveAssente:
      description: The proxy has no key configured for that provider
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example:
            errore: "Il proxy non ha una chiave per groq. Impostala con: wrangler secret put
GROQ_KEYS"

security: []
```

```
# CLIENT authentication is deliberately absent: the access control is the
# ORIGINI list plus the per-IP limits. Adding a client-side token would mean
# writing it into a static page, that is, making it public.
```

3. External APIs consumed by the application

All **optional**: with no network the application stays fully functional on local data, and says so to the user.

Service	Endpoints used	What it is for	If it does not answer

PubChem PUG-REST	`/compound/name/{name}/property/...`
	 `/compound/smiles/{smiles}/SDF?record_type=3d`
	 `/compound/cid/{cid}/PNG`
		IUPAC name, formula, CAS, GHS pictograms, 3D conformers, 2D images	The app uses the local tabulated data and says so

RCSB PDB	`https://files.rcsb.org/download/{pdb}.pdb`	Protein structures for the 3D viewer	Explicit message in the viewer
NASA / ESA	Public images and data	Astrochemistry module	The module shows the tabulated data
AI providers	At the user's choice, or via the proxy	Assistant answers	The assistant declares it and offers a reachable alternative
Crossref REST	`https://api.crossref.org/works/{doi}`	A work's record for citations (APA 7, IEEE, ISO 690, BibTeX) in the Resources and bibliography sections	Says so, with the request address; the hand-filled form remains
The app's own site	`./sw.js?controllo=<time>` with `cache: 'no-store'`	«Check for updates»: the `var CACHE = 'bsi-vNNN'` line is the published version	The GitHub fallback is tried
GitHub (fallback)	`https://raw.githubusercontent.com/samupropio1-ship-it/BioSpecInfo-v11/main/sw.js`	Same reading, only if the own site does not answer	«Cannot check now», the app keeps working offline

None of these calls transmits personal data. There are no persistent identifiers and no active telemetry (`BSI_TELEMETRY_URL` is empty).

> This is not asserted from the architecture alone: `audit_rete` seeds a canary value into the user's own data, uses the application across 6 pages and 87 sections, and inspects the URL, headers and body of every outgoing request.

---

4. Internal JavaScript APIs

Extension points for whoever has to work on the code. They are globals on `window` because the application is a set of classic scripts, not ES modules.

`window.BSISpettri` — spectral engine

Function	Signature	Description
`gruppi`	`(smiles, callback)`	Recognises functional groups via SMARTS/RDKit. Calls back with `null` if RDKit is unavailable: the caller decides whether to fall back or declare that it cannot compute.
`gruppiSync`	`(smiles) → object | null`	Synchronous version: answers only if the molecule is already cached
`bandeDaGruppi`	`(groups) → band[]`	Table of IR bands `{n, pk, w, i, f}`
`svgIR`	`(bands, options) → string`	IR spectrum in transmittance, SVG, deterministic
`legendaIR`	`(bands) → string`	Assignments in HTML
`quandoPronto`	`(callback)`	Notifies when the analysis of a molecule is available, so it can be redrawn
`precarica`	`()`	Warms up RDKit while nobody is waiting

`window.bsiProxy` — proxy state

Function	Description
`url()`	Configured address, empty string if absent
`stato()`	Promise: queries `/stato` and records which providers are covered
`copre(providerId)`	Whether that provider is covered by the proxy
`ricarica()`	Re-reads the configuration

Other notable points

Symbol	Description
`window.bsiLoadRDKit(cb)`	Loads RDKit once only and calls back with the instance
`window.irBandList(smiles)`	IR bands — replaced by the new engine, keeps the original signature

Symbol	Description
`window.makeIRsvg(bands, colour)`	IR drawing — likewise
`window.nmrPeakList(smiles)`	^{1}H peaks with integration
`window.bsiCancellaDati(groups)`	Selective deletion of the saved data
`window.bsiOpenUpdate()`	Opens the updates window
`window.bsiAccentoLeggibile(colour, background, threshold)`	Adjusts an accent so it reaches the WCAG threshold on that surface, preserving the hue
`window.bsiEtichettaLeggibile(background, threshold)`	The dual: picks the text colour from the background and, if neither extreme suffices, darkens the background while preserving the hue

`window.BSIAggiorna` — update check (`bsi-aggiorna.js`)

Function	Signature	Description
`controlla`	`() → Promise<'aggiornata' | 'nuova' | 'offline'>`	Compares the version in use with the published one and says so in a pop-up notice (`#bsi-agg-esito`). If the version differs it asks the service worker to update (`registration.update()`). It always answers
`versioneInUso`	`() → Promise<string | null>`	Asks the active service worker for its cache name with the `{ type: 'BSI_VERSIONE' }` message over a `MessageChannel`; `null` without a service worker or after 1.5 s without an answer
`versionePubblicata`	`() → Promise<string | null>`	Downloads `./sw.js?controllo=<time>` with no cache and reads its `var CACHE = '…'`

The service worker answers the `BSI_VERSIONE` message with `{ cache: CACHE }` on the port it received. The 🔄 Updates window uses these functions and, if the own site does not answer, reads the copy of `sw.js` on GitHub.

`window.BSIAttrezzi` — quick tools of the ✨ menu (`bsi-attrezzi.js`)

Symbol	Signature	Description
`apri`	`(key, subject) → boolean`	Opens the tool in a window above the page. Keys: `calcolatrice`, `derivata`, `grafico`, `costante`, `unita`, `formula`, `glossario`, `quiz`. `false` if the key does not exist
`chiudi`	`()`	Closes the window
`STRUMENTI`	object	Icon, name, description and opening function of each tool
`PER_MATERIA`	object	Which tools the ✨ menu shows for `fisica` and `matematica` (none for `chimica`)
`calcolo.calcola`	`(text) → { esatto, html, valore } | { errore }`	Exact computation with `BSIMatCAS`

Symbol	Signature	Description
`calcolo.derivaTesto`, `calcolo.integraTesto`	`(text, x?) → { testo, html } | { errore }`	Symbolic derivative and antiderivative
`calcolo.campioni`	`(text, a, b, n)`	Points of a function of x for the graph; `null` without the symbolic engine
`calcolo.cerca`	`(list, q, fields)`	Accent-insensitive search
`calcolo.quizLampo`	`(subject, n, seed)`	Questions drawn from the study kit with a reproducible generator; `null` without a kit

`window.BSIStudio` — study-kit engine (`bsi-studio.js`)

Symbol	Signature	Description
`registra`	`(subject, data) → boolean`	Merges a subject's data (`argomenti`, `riassunti`, `quiz`, `flash`, `formule`, `glossario`) and creates the five sections `s<pre>riass`, `s<pre>quiz`, `s<pre>flash`, `s<pre>form`, `s<pre>gloss` (`pre`: `fis`, `mat`, `chi`). A data file loaded before the engine queues itself in `window.BSIStudioCoda`
`dati`	`(subject) → object | null`	The subject's merged data
`materie`	`() → string[]`	The registered subjects
`sezioni`	`(subject) → string[]`	The ids of the five sections
`valida`	`(data, opts) → string[]`	The problems found, in Italian; an empty list means «fine». With `{ quantita: true }` it also checks the minimums in `MINIMI`
`MINIMI`	object	`{ riassunti: 60, quiz: 300, flash: 250, formule: 200, glossario: 250 }`
`pulisci`	`(html) → string`	Keeps only `p, b, i, ul, li, sub, sup, br`, without attributes
`norm`	`(s) → string`	Text without tags and accents, lower case
`calcolo.mulberry32`	`(seed) → () → number`	Reproducible generator in [0, 1)
`calcolo.mescola`	`(arr, rnd?) → array`	Fisher–Yates on a copy
`calcolo.leitner`	`(state, id, correct) → state`	Correct: one box up (at most 5); wrong: box 1. Returns a new object
`calcolo.punteggio`	`(answers) → { giuste, totale, perc, perArg }`	Overall and per-topic score

`window.BSISpectraAndroide` — Spectra's face (`bsi-spectra-androide.js`)

Symbol	Signature	Description
`disponibile`	`() → boolean`	Whether the browser offers WebGL

Symbol	Signature	Description
`crea`	`(container) → Promise<Androide>`	Loads three.js, `GLTFLoader` and `models/spectra-noir.glb`, and draws the face in the container
`foto`	`(px) → Promise<string>`	A PNG image of the face (data URL), for the icons
`emozioneDi`	`(sentence) → string`	A sentence's emotion from keywords; `'serena'` with no signal
`EMOZIONI`	object	The 17 emotions, each as weights of the expressive fields plus eye light and head pose
`CAMPI`	string[]	The 11 expressive fields
`COLORI`	object	Eye colour per state; `attesa: 0x1a3050`
`URL`, `BOCCA`, `versione`	—	Model path, mouth geometry, module version

The object returned by `crea` has, among others, `stato(s)` (`'attesa'`, `'ascolta'`, `'pensa'`, `'parla'`), `dici(text)` (the mouth follows the text), `allinea(i)`, `taci()`, `espressione(name, seconds, intensity) → boolean`, `emozione() → string`, `sente(sentence)` (reacts to a user sentence and passes it on to the mind), `passo(dt)`, `ferma()`, `libera()`. Since version 3.0 of the module (v238) also `concentra(c)` (target concentration between 0 and 1), `concentrazione() → number` (the current one), `addormentato() → number` (0 awake, 1 asleep), `aperture() → [left, right]` (visible opening of the two eyelids, 1 = at rest) and the `coloreOcchi` property (current eye tint, `#rrggbb`).

`window.BSISpectraRobot` (`bsi-spectra-robot.js`) exposes `disponibile`, `crea` and `foto` with the same signatures: it uses the android and, if the model does not arrive, the head built in three.js. `window.BSISpectraVolto` (`bsi-spectra-volto.js`) is the last-resort SVG portrait.

`window.BSISpectraMente` — Spectra's mind (`bsi-spectra-mente.js`)

Function	Signature	Description
`ricorda`	`(question, outcome)`	Records an episode (time, subject, section, question cut to 160 characters) and updates curiosity, attention and bond
`sente`	`(emotion, who)`	Moves the mood (and the care) according to the emotion detected in a sentence
`stato`	`() → { curiosita, attenzione, premura, umore, legame }`	The internal state, values between 0 and 1
`fondo`	`() → string`	The face's background emotion when nobody is speaking
`profilo`	`() → object`	Favourite sections, minutes per subject, favourite hour, neglected subject, quiz mistakes, recent topics
`pensieri`	`() → [{ peso, testo }]`	Suggestions ordered by weight
`saluto`	`() → string`	Greeting that takes the time and the last visit into account

Function	Signature	Description
`coscienza`	`() → string`	The summary added to the question sent to the model; empty without memory. Also exposed as `window._bsiMente`
`cosaRicordi`, `comeStai`	`() → string`	The answers to the «what do you remember» and «how are you» commands
`dimentica`	`()`	Clears the memory and removes the `bsi-spectra-mente-v1` key

It is a simulation of behaviour: the functions read and write numbers and text in `localStorage`, nothing else.

`window.BSISpectraLive` — the console (`bsi-spectra-live.js`), v238 additions

Function	Signature	Description
`difficolta`	`(text) → { valore, livello, nome }`	Difficulty of the question between 0.05 and 1, level 0–3 and its name (high school … PhD)
`livelloSezione`	`(id) → number`	First level of the `bsi-percorsi.js` path in which the section appears; −1 if none
`navigatore`	`(subject?)`	Opens (or closes) the 🧭 navigator; with a subject, shows that one
`provaCreatore`	`(active)`	Puts the console field in proof mode (password type for one minute) or takes it out
`verificaCreatore`	`(phrase)`	Checks the phrase with `BSISpectraCreatore.verifica` and answers in the dialogue, without recording the phrase

`window.BSISpectraCreatore` — the creator (`bsi-spectra-creatore.js`)

Symbol	Signature	Description
`NOME`, `NOME_BREVE`	string	`'Samuele Pio Provenzano'`, `'Samuele'`
`riconosciuto`	`() → boolean`	Whether the creator has been recognised on this device
`verifica`	`(phrase) → Promise<{ ok, rimasti?, attendi? }>`	SHA-256 fingerprint of the phrase compared with the File Manager's; after three errors `attendi` (seconds) for one minute. The phrase is not stored
`dimentica`	`()`	Removes `bsi-spectra-creatore-v1`
`visita`	`()`	Updates the last-visit date of the recognised creator
`saluto`, `chiTiHaCreato`	`() → string`	The greeting by name and the answer to «who created you?»
`contesto`	`() → string`	The `[CREATORE]` line added to the model's context

`window.BSILezione` — the guided lesson (`bsi-lezione.js`, v239)

Symbol	Signature	Description
`avvia`	`(section?, { muta? }?) → boolean`	Builds the lesson for the given section (or the visible one) and starts it in the player; `muta: true` starts with the voice off. `false` if no section is visible. A lesson already running is closed
`chiudi`	`()`	Stops the lesson, puts the control being demonstrated back as it was, removes panel and spotlight, makes the floating buttons visible again
`costruisci`	`(section?) → { sezione, titolo, capitoli, passi } | null`	Construction only, without the player. `capitoli`: «Introduzione», one name per tab (or «Esploriamo»), «Verifica» if there is a question, «Riepilogo» (English labels when the interface is in English). `passi`: objects with `tipo` among `titolo`, `scheda`, `leggi` (placeholder replaced by the tab's steps when the tab is opened), `testo`, `tela`, `cursore`, `scelta`, `quiz`, `fine`, and `cap` (chapter index)
`vai`	`(i)`	Goes to step `i` like a tap on ◀ ▶ (interrupts the voice)
`stato`	`() → { i, passi, tipo, capitoli, auto } | null`	Current step, number of steps, step type, chapters, whether auto-advance is on; `null` with no lesson
`versione`	string	`'1.0'`

Spectra Live's local command «fammi lezione» («give me a lesson») in `BSISpectraLive.comando` calls `BSILezione.avvia()` after closing the console.

`window.BSISpectraVoce` — the voice (`bsi-spectra-voce.js`, v239)

Symbol	Signature	Description
`qualita`	`() → 'audio' | 'sistema' | 'testo'`	`'audio'` with an OpenAI key in `bsi_api_keys` and Web Audio; `'sistema'` with `speechSynthesis` only; `'testo'` without voice
`parla`	`(text, { suBocca?, timbro?, soloSistema? }?) → Promise<{ ok, motore, durata? } | { annullato }>`	In audio mode sends the text (up to 3,800 characters) to `api.openai.com/v1/audio/speech` (`gpt-4o-mini-tts`, voice «onyx»; `tts-1` as fallback), runs it through the chain and calls `suBocca({ energia, centroide, apertura, rotondo, largo, frazione })` every frame; if the request fails, or with `soloSistema`, uses `speechSynthesis`. Interrupts the previous sentence
`ferma`	`()`	Stops the audio and the browser's synthesis
`catena`	`(audioContext, timbre?) → { ingresso, uscita, analizzatore, ferma }`	The effects chain of `TIMBRO`
`impulso`	`(audioContext, seconds) → AudioBuffer`	Synthetic reverb impulse response (stereo)
`leggiBocca`	`(analyser, data) → { energia, centroide, apertura, rotondo, largo }`	Lip sync from audio: RMS energy and spectral centroid

Symbol	Signature	Description
`TIMBRO`	object	`bassiHz` 140, `bassiDb` 5, `fangoHz` 350, `fangoDb` −3, `presenzaHz` 2800, `presenzaDb` 3, `anelloHz` 55, `anello` 0.12, `pettineMs` 4.3, `pettineRetro` 0.35, `pettine` 0.18, `calore` 1.6, `stanzaS` 1.4, `stanza` 0.16, `volume` 0.95

`window.BSISpectraCorpo` — the body (`bsi-spectra-corpo.js`, v239)

Symbol	Signature	Description
`crea`	`(root, { animazioni?, capelliNeri? }?) → Corpo`	Attaches the driver to a three.js object with a skeleton; with `animazioni` (the GLB's clips) it recognises `idle`, `walk`, `sit`, `stand`, `wave`, `point`, `talk`; the hair turns black unless `capelliNeri: false`
`manichino`	`({ neon? }?) → THREE.Group`	The test mannequin (since v240 only for the driver's bench, never as Spectra): bones with Mixamo names in T-pose, head with 26 ARKit blendshapes, separate eyes, red hair cap
`EMOZIONI`, `OSSA`, `normaNome`	objects, function	ARKit weights per emotion; regular expressions for bone names; name normalisation

Methods of a `Corpo`:

Method	Signature	Description
`rapporto`	`() → { ossa, mancanti, blendshape, capelli, clip, locomozione, viso }`	What the model can do (the requirements check of `docs/spectra/01`)
`capelliNeri`	`(colour?)`	Colour (default `#0a0a0c`), roughness, and texture desaturated in the shader
`orienta`	`(bone, direction, weight)`	Brings the bone → child direction onto a world direction
`emozione`	`(name, seconds?, intensity?) → boolean`	One of the emotions in `EMOZIONI` for a few seconds
`dici`, `audio`, `taci`	`(text)`, `(mouthState)`, `()`	Lip sync from text, from audio (`suBocca` of `BSISpectraVoce`), stop
`guarda`	`(point)`	Eyes, neck and head towards a point
`fai`	`(gesture, target?, seconds?)`	`'indica'` (point), `'spiega'` (explain), `'pensa'` (think), `'attende'` (wait), `'conclude'`
`cammina`, `siedi`	`(point)`, `(yes?)`	Walk to a point; sit (`false` to stand up)
`peso`, `imposta`	`(name) → number | null`, `(name, value) → boolean`	Reads and writes a blendshape weight
`passo`	`(dt)`	Advances the behaviour by `dt` seconds; to be called every frame

`window.BSISpectraFigura` — Spectra with the body (`bsi-spectra-figura.js`, v240)

Symbol	Signature	Description
`Spectra`	`new (T, scene, renderer, opz?) → controller`	Spectra's body in an existing scene. Order: IndexedDB `bsi-spectra-modelli` → `models/spectra-corpo.glb` (development only) → `models/spectra-cyberpunk.glb` → no body (the mannequin only with `opz.manichino === true`, for the driver's bench). With `opz.soloDispositivo` IndexedDB only: the figure from `monta` enables it by default (in the app no requests for the development files, no 404s); the studio still tries the files. Rig with `bsi-spectra-rig.js` (`RIG`), «Noir» face on the collar, «solidale»
`monta`	`(container, opz?) → figure`	Lab-style frame with Spectra standing, HUD «SPECTRA · IN ASCOLTO / PENSA / PARLA», buttons «🧍 Corpo», «🎙️ Parla con me», «🎓 Studio», invitation when the body is missing; listens to `bsi-spectra-stato`, `bsi-spectra-risposta`, `bsi-spectra-dice`
`sezioni`	`(id[]) → figure`	One shared figure (one WebGL context) moved into the open AI section (`sai`, `sareachim`, `sareafis`, `sareamat`); the 3D files load on first opening
`prepara`	`() → Promise`	Loads three.js, the GLB loader, the body driver and the rig if they are missing
`dbCorpo`	`('leggi' | 'metti' | 'togli', buffer?) → Promise`	The `.glb` body in IndexedDB (`bsi-spectra-modelli`, store `file`, key `corpo`)
`emozioneDi`	`(text) → string`	The emotion suggested by an answer's content
`Ologramma`	`new (T, scene) → hologram`	The holographic screen to Spectra's right; methods `costruisci(name)`, `mostra(name)`, `passo(dt)`; scenes computed every frame, plus `'schermo'` (the section behind the window, re-rendered with `BSISpectraLive.cattura` 5 times a second; a section without graphs is projected as a card: title, tab, up to 6 controls with values, visible text)
`SCENE_OLO`	object	The six scenes (`pianeti`, `molecola`, `spettro`, `superficie`, `orbitale`, `onda`): for each `nome` (it, en), `sez` (the id of the section the label opens) and `parole` (the regular expression that selects it)
`sceneDi`	`(text) → string | null`	The scene matching the words of a text, or `null`
`LIMITE`, `RIG`, `figura`	number, object, figure	60 MB; the rig parameters for the author's body; the sections' shared figure (or `null`)

Controller methods: `carica()`, `daFile(file)`, `daBuffer(buffer)`, `usaScena(scene)` (for benches), `passo(dt)`, `emozione(name, seconds?)`, `dici(text)`, `taci()`, `fai(gesture, target?, seconds?)`, `presente() → boolean`.

New event: **`bsi-spectra-dice`** (`CustomEvent` on `document`, `detail: { testo }`), dispatched by `parla()` in `bsi-spectra-live.js` and by `bsi-assistente.js` when it solves an exercise; the figure says it with lip sync and a gesture.

New event: **`bsi-spectra-tema`** (`CustomEvent` on `document`, `detail: { testo, schermo }`), dispatched by `bsi-spectra-spiega.js` with the question (and when the tab opens with a section behind); the hologram picks the scene with `sceneDi(text)`, or shows the screen live if `schermo` is true. When answering, the figure first points at the hologram («indica» gesture, about 2.5 s) and then accompanies with «spiega».

`window.BSISpiega` — «🧠 Spiega» (`bsi-spectra-spiega.js`, v240)

Symbol	Signature	Description
`costruisci`	`(panel)`	Builds the «🧠 Spiega» tab in the given panel (called by `bsi-ai-hub.js`): question, level, «📺 Includi lo schermo» checkbox with the screen card, quick prompts, «↺ Nuova spiegazione», the «🗂️ Tutte le sezioni nell'ologramma» card (search, subject filter, «🎞️ Mostrami tutto»)
`catalogo`	`() → [{ id, nome, gruppo, g, materia }]`	Every section of the app bar (`.nav-group-panel [data-s]`), with its group label and subject (`BSIMaterie.materiaDi`; `'comune'` if none): 194 sections (chemistry 75, physics 60, mathematics 30, shared 29). Read from the bar, not written by hand
`sezioneDi`	`(text) → catalogue entry | null`	The section a question names: meaningful words of the names (with stems), acronyms (MS, IR, UV, NMR, MO…) weighted more, synonyms («spettro di massa» → MS, «infrarosso» → IR, «risonanza magnetica» → NMR, «orbitali molecolari» → MO); ties go to the shorter name. E.g. «titolazioni» → `stitr`, «integrali per parti» → `smatcalc`, «ciao come stai» → `null`
`mostraSezione`	`(id) → Promise<boolean>`	Opens the section behind the Spectra window (`BSISpectraLive.calcolo.apriSezione`, which switches subject if needed) and refreshes the screen, so the hologram shows it live
`giro`	`(subject?)`	«🎞️ Mostrami tutto»: every section of the subject (`'chimica'`, `'fisica'`, `'matematica'`; all if empty) in the hologram, one every 7 s, with the counter «n/N · name»
`giroFerma`	`()`	Stops the tour
`chiedi`	`(question, { schermo?, livello? }?) → Promise`	Sends the question to the chat's model; with `schermo` it adds the screen text and, in the first turn, the image; without `schermo`, if the question names a section (`sezioneDi`) other than the open one, it first opens it (`mostraSezione`) and then asks with the screen, unless the user unchecked the box (`ST.schermoNo`): then only the question; follow-up questions continue the conversation. Without a model it shows what the app knows on its own
`schermo`	`() → { testo, tela, svg, grafici, titolo, materia }`	Reads the open section: title, tab, controls with value and range, visible tables (up to 4 × 40 rows), visible text, visible canvases and SVGs
`immagine`	`(S) → Promise<dataURL>`	Composes into a JPEG (at most 1400 px wide) the visible canvases (`BSILab3D` scenes re-rendered first) and the visible SVGs rasterised
`sistema`	`(level, subject) → string`	The system prompt: level, `PROFONDITA`, mandatory structure, rigour

Symbol	Signature	Description
`aggiornaSchermo`	`()`	Refreshes the screen card (title, number of graphs and 3D scenes, thumbnail)
`LIVELLI`, `PROFONDITA`	array	The five levels (High school to Advanced PhD · frontier) and what is asked at each
`stato`	`() → { turni, livello, inCorso }`	The state of the conversation

`window.bsiLLM` — the chat's provider (`bsi-ai-hub.js`, v240)

Symbol	Signature	Description
`provider`	`() → string`	The provider chosen in the Spectra window (or the saved one)
`chiave`	`(provider) → string | null`	The key to use for that provider, as the chat does; used by «🧠 Spiega»

`window.BSIStudioLezioni` and `window.SpectraStudio` — the studio (v239)

`BSIStudioLezioni` (`bsi-spectra-studio-lezioni.js`, also usable with `require` in node) exposes `LEZIONI` (for each lesson `id`, `materia`, `predici`, `testa`, `dimostra`, `rivela`), `SCENE` (each lesson's 3D objects) and `calcolo`: `caduta(h, v0, g?, dt?, k?)`, `vsepr(nBonds, nLonePairs, wLP?, iter?)`, `parziale(N)`, `armonica(N)`, `coda(N)`, `fourierX2(x, N)`, `mcd(a, b)`, `coprimi(M, max, seed?)`.

`SpectraStudio`, defined inside `spectra-studio.html`, serves the benches: `apri(id)`, `scegli(i)`, `testa()`, `dimostra(k)`, `rivela()`, `stato() → { lez, fase, misure, previsione, scena, volto }`.

`window.BSISpectraAndroide.crea` — the face in someone else's scene (v239)

`crea(parent, { scena, renderer, senzaBusto }?)`: with `scena` the face lives in an existing three.js scene, without its own renderer, lights, camera or drawing loop (the host calls `passo(dt)` and attaches `radice` to the body); with `senzaBusto` the model's neck and shoulders are hidden. The object also exposes `parti` (the model's meshes by name).

Document updated to version `bsi-v240`.